Phytium飞腾

# 飞腾派 OpenHarmony
# 用户指导手册

（V1.0）

飞腾信息技术有限公司
www.phytium.com.cn
2023 年 08 月 24 日

**版权所有 © 飞腾信息技术有限公司 2023。 保留一切权利。**

未经本公司同意，任何单位、公司或个人不得擅自复制、翻译、摘抄本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。

**商标声明**

Phytium 和其他飞腾商标均为飞腾信息技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

**特别提示**

本文档仅作为使用指导，飞腾对本文档内容不做任何明示或暗示的声明或保证。本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

由于产品版本升级或其他原因，本文档内容会不定期进行更新，如有变更，恕不另行通知。

**最新技术资源**

飞腾嵌入式软件开源社区 https://gitee.com/phytium_embedded

或者访问飞腾软件开发者平台 https://service.phytium.com.cn/developer/

# 目录

1 简介 ..... 3
1.1 文档目的 ..... 3
1.2 飞腾派芯片简介 ..... 3
1.3 OpenHarmony 简介 ..... 3
1.4 Phytiumpi OpenHarmony 开源仓库简介 ..... 3
2 搭建开发环境 ..... 4
2.1 交叉编译硬件环境 ..... 4
2.2 下载 repo 脚本文件 ..... 4
2.3 获取 OpenHarmony 标准系统源码 ..... 4
2.4 获取编译工具链 ..... 4
2.5 执行 prebuilts ..... 5
3 代码下载与整合 ..... 5
3.1 下载 phytium OH_3.2_release phytiumpi-firefly 适配源码 ..... 5
3.2 整合 phytiym device vendor 相关源码 ..... 5
4 代码编译 ..... 6
4.1 OpenHarmony 编译 ..... 6
5 镜像烧写 ..... 6
5.1 SD 卡分区 ..... 7
5.2 system/vendor/userdata 分区 ..... 7
5.3 boot 分区 ..... 7
5.3.1 Uboot 启动 ..... 7
5.4 将 Uboot 镜像烧写到 SD 卡 ..... 8
6 Generate Image 工具 ..... 8
6.1 文件说明 ..... 8
6.2 使用方法 ..... 9
7 设备启动 ..... 9
7.1 Uboot 启动 ..... 9
8 维护者邮箱 ..... 10

A. 更新记录 ..... 10

# 1 简介

## 1.1 文档目的

该文档主要介绍，如何在飞腾信息技术有限公司的 phytiumpi-firefly 开发板平台上运行 OpenHarmony 标准系统。

## 1.2 飞腾派芯片简介

飞腾派是飞腾嵌入式四核处理器，该处理器兼容 ARM V8 指令集，包含 2 个 FTC664 核和 2 个 FTC310 核，其中 FTC664 核主频可达 1.8GHz，FTC310 核主频可达 1.5GHz。

## 1.3 OpenHarmony 简介

OpenHarmony 是由开放原子开源基金会（OpenAtom Foundation）孵化及运营的开源项目，目标是面向全场景、全连接、全智能时代，搭建一个智能终端设备操作系统的框架和平台，促进万物互联产业的繁荣发展。OpenHarmony 支持轻量系统、小型系统、标准系统三种类型系统。OpenHarmony 相关具体内容可以参考 OpenHarmony 官网以及 OpenHarmony 码云仓库介绍。

## 1.4 Phytiumpi OpenHarmony 开源仓库简介

Phytium 在飞腾派上对 OpenHarmony 3.2Release 标准系统已完成适配，项目代码已在码云 Gitee 上开源，开源仓库地址：Phytium-OpenHarmony-Phytiumpi。

该项目基于 phytiumpi-firefly 开发板开发，支持 Linux kernel 5.10，并已使能飞腾派内置提供的视频解码硬件加速。

代码仓库主要目录说明如下：

```
├── device_board_phytium     #飞腾开发板代码仓库
├── device_soc_phytium       #飞腾芯片代码仓库
├── phytium_env.sh           #自动化合 patch 脚本
├── vendor_phytium           #飞腾开发板方案定制代码仓库
└── ...
```

# 2 搭建开发环境

## 2.1 交叉编译硬件环境

工欲善其事必先利其器，所以首先准备一台装有 ubuntu20.04 系统 X86 主机，内存最低配置要求 16G。硬件配置肯定是多多益善了，配置越高，编译过程就更加的纵享丝滑了！

## 2.2 下载 repo 脚本文件

1. 注册码云 gitee 账号。
2. 注册码云 SSH 公钥，请参考码云帮助中心。
3. 安装 git 客户端和 git-lfs 并配置用户信息。

```
git config --global user.name "yourname"
git config --global user.email "your-email-address"
git config --global credential.helper store
```

4. 安装码云 repo 工具，可以执行如下命令。

```
curl -s https://gitee.com/oschina/repo/raw/fork_flow/repo-py3 > /usr/local/bin/repo
chmod a+x /usr/local/bin/repo
pip3 install -i https://repo.huaweicloud.com/repository/pypi/simple requests
```

## 2.3 获取 OpenHarmony 标准系统源码

推荐版本 OpenHamony 3.2 release。

通过 repo + ssh 下载（需注册公钥，请参考码云帮助中心）。

```
export WORK_SPACE=/home/xxx/OpenHarmony #替换成自己定义的 workspace 路径
export PROJ_ROOT=$WORK_SPACE/3.2-release
mkdir $WORK_SPACE
mkdir $PROJ_ROOT
cd $PROJ_ROOT
repo init -u git@gitee.com:openharmony/manifest.git -b OpenHarmony-3.2-Release
--no-repo-verify
repo sync -c
repo forall -c 'git lfs pull'
```

## 2.4 获取编译工具链

使用安装包方式获取编译工具链。

```
sudo apt-get update && sudo apt-get install binutils git git-lfs gnupg flex bison gperf
build-essential zip curl zlib1g-dev gcc-multilib g++-multilib libc6-dev-i386
lib32ncurses5-dev x11proto-core-dev libx11-dev lib32z1-dev ccache libgl1-mesa-dev
libxml2-utils xsltproc unzip m4 bc gnutls-bin python3.8 python3-pip ruby
```

### 2.5 执行 prebuilts

在源码根目录下执行脚本，安装编译器及二进制工具。

```
cd &PROJ_ROOT
bash build/prebuilts_download.sh
```

## 3 代码下载与整合

### 3.1 下载 phytium OH_3.2_release phytiumpi-firefly 适配源码

创建存放 phytiym phytium OH_3.2_release 源码的目录，进到该目录，下载 phytium OH_3.2_release phytiumpi-firefly 适配源码：

```
export PHY_DEV=$WORK_SPACE/phytium_device
mkdir $PHY_DEV
cd $PHY_DEV
git clone https://gitee.com/phytium_embedded/phtyium-openharmony-phytiumpi-device.git
```

### 3.2 整合 phytiym device vendor 相关源码

通过前面的步骤，我们下载好了相关飞腾部分的修改。是时候将其和入到鸿蒙主干了。这里我们贴心的为开发者准备了一键构建脚本，我们这里执行脚本。

```
./phytium_env.sh    $PROJ_ROOT
THE OHOS_PATH_ROOT /home/xxx/OpenHarmony/3.2-release
#### sync phytium env start!    ####
#### sync device_soc_phytium    ####
#### sync device_soc_phytium end    ####
#### sync device_board_phytium    ####
#### sync device_board_phytium end    ####
#### sync vendor_phytium    ####
#### sync vendor_phytium end    ####
#### sync kernel    ####
#### sync kernel end    ####
#### applay 0001_drivers_peripheral.patch    ####
#### applay 0001_foundation_arkui_ace_engine.patch    ####
```

```
#### applay 0001_foundation_graphic_graphic_2d.patch    ####
#### sync phytium end!    ####
```

**注意：这里我们需要传入的参数是鸿蒙的根目录！**

# 4 代码编译

## 4.1 OpenHarmony 编译

```
cd $PROJ_ROOT
./build.sh --product-name phytiumpi-firefly --ccache --target-cpu arm64
```

编译成功提示:

```
post_process
=====build phytiumpi-firefly successful.
```

编译完成，我们进入到 out 目录查看生成的文件：

```
cd out/phytiumpi-firefly/packages/phone/images/
tree
├── kernel
│   ├── dtb
│   │   └── phytiumpi_firefly.dtb
│   ├── EFI
│   │   └── BOOT
│   │       ├── bootaa64.efi
│   │       └── grub.cfg
│   └── Image
├── ramdisk.img
├── system.img
├── updater.img
├── userdata.img
└── vendor.img
4 directories, 27 files
```

可以看到，其中生成了内核和系统相关镜像文件！

# 5 镜像烧写

飞腾派开发板支持从 flash、SD 卡、EMMC 三种介质启动，通过拨码控制，具体设置方法请参考对应的飞腾派开发板使用手册。OpenHarmony 仓库代码默认是从 SD 卡启动，这里分区和烧录以 SD 卡举例。 修改成其他介质启动，需要修改代码，参考《OpenHarmony 多种存储介质配置说明》。

**如果使用 SD 卡启动，也可以使用[Genarate_Img 工具](章节 6)简化烧录流程。**

## 5.1 SD 卡分区

为了兼容从 SD 卡启动，前面会预留 64M 空间用于存放 uboot 固件，第一个分区的起始块偏移为 131072。由于不是从首地址开始分区，分区时都要手动指定起始地址，第一个分区地址地址为 131072，后面分区起始地址是前一个分区结束地址基础上再加 1。

在 linux 环境下，使用 fdsik 命令对 SD 卡进行分区，创建 4 个分区，依次为 boot、system、vendor、userdata，根据实际情况设定，比如可以设定为 500M、2G、500M、2G，分区参数可参考如下配置。

```
Device     Boot   Start      End Sectors  Size Id Type
/dev/sdx1          131072   1155071 1024000   500M 83 Linux
/dev/sdx2         1155072   5349375 4194304     2G 83 Linux
/dev/sdx3         5349376   6373375 1024000   500M 83 Linux
/dev/sdx4         6373376  10567679 4194304     2G 83 Linux
```

**注意：fidsk 命令详细使用方法可自行百度谷歌，或者参考飞腾嵌入式 LINUX 用户 手册中的磁盘分区部分内容。**

## 5.2 system/vendor/userdata 分区

将这三个分区并格式化为 ext4，sdX 中的 X 烧写之前先用 df 命令确认一下是多少，可能是 b，c…等，**这里一定要注意不要烧错。**

```
sudo mkfs.ext4 sdx2
sudo mkfs.ext4 sdx3
sudo mkfs.ext4 sdx4
```

使用 dd 命令将 4.1 OpenHarmony 编译章节编译生成的镜像文件烧写到对应分区中。

```
sudo dd if=system.img of=/dev/sdX2 bs=1M
sudo dd if=vendor.img of=/dev/sdX3 bs=1M
```

**重要的事情说三遍，不要烧录 userdata 分区！只需要将 sdx4 格式化为 ext4 格式就 OK 了！**

## 5.3 boot 分区

boot 分区的烧录，启动方式不一样，烧录方法不一样，区分为 Uboot 启动和 UEFI 启动两种，推荐使用 Uboot 启动。

### 5.3.1 Uboot 启动

1. 将第一个分区格式化为 ext4

```
sudo mkfs.ext4 sdx1
```

2. 挂载 boot 分区

```
mkdir ~/disk
sudo mount /dev/sdx1 ~/disk
```

3. 将前面生成的 Kernel 镜像，设备树文件，以及 ramdisk 拷贝到 boot 分区。

```
sudo cp &PROJ_ROOT/out/phytiumpi-firefly/packages/phone/images/kernel/Image ~/disk/
sudo cp &PROJ_ROOT/out/phytiumpi-firefly/packages/phone/images/kernel/dtb/phytiumpi_firefly.dtb
~/disk/
sudo cp &PROJ_ROOT/out/phytiumpi-firefly/packages/phone/images/ramdisk.img ~/disk/
sync
```

4. 卸载 boot 分区

```
sudo umount ~/disk
```

### 5.4 将 Uboot 镜像烧写到 SD 卡

如果板子不是设成从 SD 卡启动，这小节可以跳过。

当板子设成从 SD 卡启动时，首先将分区表信息保存为 table.bin，然后将从飞腾派萤火工厂获取到的 Uboot 镜像<uboot.bin>烧写到 SD 卡，最后将 table.bin 再重新写回 SD 卡。

在 device_board_phytium/phytiumpi-firefly/tools/uboot/phytiumpi/目录下有包含萤火工厂提供的 sd 卡 uboot 可以使用，版本号为 U-Boot 2022.01-v1.32-g457bb7da。

**请注意，该固件只能用于从 SD 卡引导，该项目不维护 uboot，最新 uboot 稳定版本请联系飞腾派萤火工厂获取。**

```
sudo dd if=/dev/sdx of=table.bin bs=1 skip=446 count=66
sudo dd if=<uboot.bin> of=/dev/sdx
sudo dd if=table.bin of=/dev/sdx bs=1 seek=446 count=66
```

烧录完成后，跳转到[设备启动](章节 7)，上电启动。

## 6 Generate Image 工具

为简化烧录流程，可以使用 Generat_Image 工具，将所有需要烧录的镜像文件全部打包成一个文件，然后直接烧录到 SD 卡。

### 6.1 文件说明

```
device_board_phytium/phytiumpi-firefly/tools/generate_image
├── generate_image.sh     #镜像打包脚本
├── part.cfg              #分区配置文件，主要用于配置各分区大小
└── uboot
    └── phytiumpi         #phytiumpi SD 卡引导固件
```

### 6.2 使用方法

1. 确保你的账户有 sudo 权限，并安装打包工具依赖的系统工具。

```
sudo apt install dosfstools parted kpartx
```

2. OpenHarmony 编译时，编译脚本会将 generate_image 下所有内容拷贝到镜像输出目录：

```
out/phytiumpi-firefly/packages/phone/images
```

请确保相关镜像文件(Image、dtb 文件、ramdisk.img、system.img、vendor.img)已经生成，以及 generate_img 脚本已经包含，进入该目录。

```
cd out/phytiumpi-firefly/packages/phone/images
```

如果检查没有，或者想打包其他镜像文件，请将相关镜像文件，与 generate_image 目录下的所有文件，放到同一目录下，并进入该目录。

3. 执行 generate_image.sh 脚本等待完成打包。如果需要打包 uboot，请将 uboot 文件路径作为参数带上。这个过程需要执行账户有 sudo 权限。

```
sudo
./generate_image.sh uboot/phytiumpi/fip-all-v2.1-oh-2GB.bin
```

打包完成会有如下打印信息：

```
start creating empty image, please wait.....
3904+0 records in
...
...
losetup loopdevice.....
generate phytiumpi_oh_fip-all-v2.0-oh-2GB.img successfully!!!!!!
```

4. Linux 环境下，使用 dd 命令烧录即可.

```
sudo dd if=phytiumpi_oh_fip-all-v2.1-oh-2GB.img of=/dev/sdb bs=1M
```

也可以在 windows 环境下，使用 win32Diskimager、rufus、balenaEtcher 等烧录工具进行烧录。

## 7 设备启动

将烧写好的 SD 卡插入 SD 卡槽，板子配置成合适的启动介质，连接串口线，连接到 phytiumpi-firefly 开发板上。上位机的串口调试工具波特率设置为 115200，上电开机。

### 7.1 Uboot 启动

仓库中的 uboot，已经将正确的 uboot 启动参数配置，上电后，会自动进入系统。如果卡在 uboot 阶段，需按如下配置设置并保存 uboot 环境变量，最后输入 boot 进入系统。

```
setenv bootargs console=ttyAMA1,115200 earlycon=p1011,0x2800d000 root=/dev/ram0
```

```
elevator=deadline rootwait rw loglevel=6 hardware=phytiumpi rootfstype=ext4
initrd=0x93000000,90M cma=256M
setenv bootcmd "ext4load mmc 0 0x90100000 phytiumpi_firefly.dtb;ext4load mmc 0
0x90200000 Image;ext4load mmc 0 0x93000000 ramdisk.img;booti 0x90200000 -
0x90100000"
saveenv
boot
```

之后开机不需要再设置，会按这里保存的环境变量自动进入系统。

# 8 维护者邮箱

飞腾信息技术有限公司

zhangjianwei@phytium.com.cn

tangkaiwen@phytium.com.cn

xiayan1086@phytium.com.cn

libowen1180@phytium.com.cn

chenzigui1762@phytium.com.cn

# A. 更新记录

| 发布日期 | 作者 | 说明 |
|---|---|---|
| 2023-08-24 | 夏炎 | 初稿 |
| 2023-08-30 | 夏炎 | 1. 增加 generate_img 工具章节，简化烧录流程<br>2. 修改 git clone 地址 |